```
○ ○ ○

///<summary>
// Initialize all references and start background music coroutine
///</summary>
private void InitializeDollAI()
{
    _player = GameObject.FindGameObjectWithTag("Player");
    _animator = GetComponent<Animator>();
    _ai = GetComponent<NavMeshAgent>();
    _saveCooldownToResetPatrolling = _cooldownToResetPatrolling;

    StartCoroutine(DelayForStartBgMusic());
}

///<summary>
// Handles doll patrolling to the next patrol point
///</summary>
private void SetNextPatrolPoint()
{
    _indexOfPatrolPoint = (_indexOfPatrolPoint + 1) % _patrolPoints.Count;
    _ai.SetDestination(_patrolPoints[_indexOfPatrolPoint].position);
}

///<summary>
// Starts glitch visual/audio effect
///</summary>
private void StartGlitchEffect()
{
    _repeatCount = Random.Range(_spawned ? 1 : 2, 4);
    _spawned = true;
    StartCoroutine(DelayGlitchEffect());
}

///<summary>
// Coroutine for glitch effect flickering
///</summary>
private IEnumerator DelayGlitchEffect()
{
    for (int i = 0; i < _repeatCount; i++)
    {
        _glitchAudio.Play();
        _dollSkin.SetActive(false);
        _gameEngineScript.DollGlitchEffect = true;
        yield return new WaitForSeconds(Random.Range(0.05f, 0.2f));
        _glitchAudio.Stop();
        _dollSkin.SetActive(true);
        _gameEngineScript.DollGlitchEffect = false;
        yield return new WaitForSeconds(Random.Range(0.05f, 0.2f));
    }
    yield return new WaitForSeconds(Random.Range(5f, 15f));
    StartGlitchEffect();
}

///<summary>
// Plays scissor audio effect periodically
///</summary>
private void PlayScissorEffect()
{
    _scissorAudio.Play();
    StartCoroutine(DelayForScissorEffect());
}

///<summary>
// Coroutine for scissor effect delay
///</summary>
private IEnumerator DelayForScissorEffect()
{
    yield return new WaitForSeconds(Random.Range(15f, 25f));
    PlayScissorEffect();
}

///<summary>
// Resets doll audio sources to prevent freeze
///</summary>
private void FixFreezeForAudioSource()
{
    _lalalaAudio.Play();
    _bgMusicAudio.Play();
    _screamerAudio.Play(); _screamerAudio.Stop();
    _scissorAudio.Play(); _scissorAudio.Stop();
    _walkAudio.Play();
    _dollChaseAudio_1.Play(); _dollChaseAudio_1.Stop();
    _dollChaseAudio_2.Play(); _dollChaseAudio_2.Stop();
}

///<summary>
// Handles doll patrolling and following player
///</summary>
private void ResetPatrollingCooldown()
{
    if (_cooldownToResetPatrolling > 0)
        _cooldownToResetPatrolling -= Time.deltaTime;
    else
    {
        _patrolling = true;
        _walkAudio.pitch = 0.7f;

        if (_dollChaseAudio_1.isPlaying)
        {
            _dollChaseAudio_1.Stop();
            _dollChaseAudio_2.Stop();
            _walkAudio.Play();
            _lalalaAudio.Play();
            _bgMusicAudio.Play();

            if (_coroutineForRandomReplieIsRuning) ResetRandomDollReplies();
        }

        _ai.SetDestination(_patrolPoints[_indexOfPatrolPoint].position);
    }

    if (_seePlayer >= 0)
        _seePlayer -= Time.deltaTime;
}

///<summary>
// Coroutine to delay background music start
///</summary>
private IEnumerator DelayForStartBgMusic()
{
    yield return new WaitForSeconds(_delayForStartBgMusic);
    _bgMusicAudio.Play();
}

///<summary>
// Detect player in view radius and start follow
///</summary>
private void ScanForPlayer()
{
    Vector3 start = _scanPosition.transform.position;
    start.y -= 0.3f;

    for (int i = -_yawDetection; i < _yawDetection; i += 5)
    {
        Vector3 dir = Quaternion.Euler(0f, i, 0f) * _scanPosition.forward;
        if (Physics.Raycast(start, dir, out RaycastHit hit, _distanceDetection) &&
hit.collider.CompareTag("Player"))
        {
            _patrolling = false;
            _seePlayer = 1;
            StartFollowPlayerSFX();
            _cooldownToResetPatrolling = _saveCooldownToResetPatrolling;
            Debug.DrawRay(start, dir * _distanceDetection, Color.red);
        }
    }
}

///<summary>
// Stops background and doll sounds when chasing player
///</summary>
private void StartFollowPlayerSFX()
{
    _lalalaAudio.Stop();
    _bgMusicAudio.Stop();
    _dollReplieAudio.Stop();

    if (_coroutineForRandomReplieIsRuning)
    {
        StopCoroutine(_coroutineForRandomReplie);
        _coroutineForRandomReplieIsRuning = false;
    }
}

///<summary>
// Scan close range to catch player
///</summary>
private void ScanforCatchPlayer()
{
    Vector3 start = _scanPosition.transform.position;
    start.y -= 0.4f;

    for (int i = -100; i < 100; i += 5)
    {
        Vector3 dir = Quaternion.Euler(0f, i, 0f) * _scanPosition.forward;
        if (Physics.Raycast(start, dir, out RaycastHit hit, _catchDistance) &&
hit.collider.CompareTag("Player") && 1.gameEngineScript.CatchPlayer)
        {
            CatchPlayer();
        }
        Debug.DrawRay(start, dir * _catchDistance, Color.green);
    }
}

///<summary>
// Handles catching the player with animation, sound, and scene reset
///</summary>
private void CatchPlayer()
{
    _lalalaAudio.Stop();
    _bgMusicAudio.Stop();
    _walkAudio.Stop();
    _scissorAudio.gameObject.SetActive(false);
    _dollChaseAudio_1.Stop();
    _dollChaseAudio_2.Stop();
    _screamerAudio.Play();

    StartCoroutine(ResetScene());

    _gameEngineScript.CatchPlayer = true;
    _gameEngineScript.CatchContainer.SetActive(true);
    _gameEngineScript.SetJumpscare(1);
    _animator.SetTrigger("catch");

    _ai.speed = 0;
    _ai.angularSpeed = 0;

    CatchLook();
}

///<summary>
// Adjust doll and player look directions when catching
///</summary>
private void CatchLook()
{
    _dollSkin.transform.LookAt(_player.transform.position);
    float saveRotationY = _dollSkin.transform.localEulerAngles.y;
    _dollSkin.transform.localRotation = Quaternion.Euler(0, saveRotationY, 0);

    _player.transform.LookAt(_eye.transform.position);
    _gameEngineScript.PlayerScript.MainCameraAnimator.transform.LookAt(_eye.transform.position);
}

///<summary>
// Resets doll replies randomly
///</summary>
private void ResetRandomDollReplies()
{
    _lalalaAudioAnimator.SetBool("on", true);
    _coroutineForRandomReplie = StartCoroutine(DelayForRandomReplies());
}

///<summary>
// Coroutine to play random doll replies
///</summary>
private IEnumerator DelayForRandomReplies()
{
    _coroutineForRandomReplieIsRuning = true;
    float randomDelay = Random.Range(maxDelay * 0.7f, maxDelay);
    yield return new WaitForSeconds(randomDelay);

    _lalalaAudioAnimator.SetBool("on", false);
    yield return new WaitForSeconds(1);

    float clipDelay = SetDollReplie();
    yield return new WaitForSeconds(clipDelay + 2);
    ResetRandomDollReplies();
}

///<summary>
// Plays a random doll reply and returns its length
///</summary>
private float SetDollReplie()
{
    int clipIndex = CheckRepliesUsed();
    _dollReplieAudio.clip = _dollReplieClips[clipIndex];
    _dollReplieAudio.Play();
    SetRepliesUsed(clipIndex);
    return _dollReplieClips[clipIndex].length;
}

///<summary>
// Track used doll replies to avoid repeats
///</summary>
private void SetRepliesUsed(int index)
{
    if (_dollRepliesUsed.Count + 2 >= _dollReplieClips.Count) _dollRepliesUsed.Clear();
    _dollRepliesUsed.Add(index);
}

///<summary>
// Checks which reply clip can be used next
///</summary>
private int CheckRepliesUsed()
{
    bool canSetReply = false;
    int selectRandomClip;
    do
    {
        selectRandomClip = Random.Range(0, _dollReplieClips.Count);
        if (_dollRepliesUsed.Count == 0) break;

        for (int i = 0; i < _dollRepliesUsed.Count; i++)
        {
            if (_dollRepliesUsed[i] == selectRandomClip) break;
            if (i >= _dollRepliesUsed.Count - 1) canSetReply = true;
        }

    } while (!canSetReply);
    canSetReply = false;

    return selectRandomClip;
}

///<summary>
// Checks locker and handles doll behavior near locker
///</summary>
private void CheckLocker()
{
    float lockerDistance = Vector3.Distance(transform.position,
_gameEngineScript.LockerDollPosition.position);
    if (lockerDistance > 1.3f)
    {
        _ai.SetDestination(_gameEngineScript.LockerDollPosition.position);
    }
    else
    {
        if (_gameEngineScript && _cooldownToResetPatrolling <= 0 && _checkLockerDoor <= 0 && _seePlayer
<= 0)
        {
            _animator.SetBool("idle", true);
            _walkAudio.Stop();
            _ai.speed = 0;
        }
        else
        {
            _gameEngineScript.LockerControllerScript.CatchPlayer();
        }
    }
}

///<summary>
// Coroutine to reset scene after player is caught
///</summary>
private IEnumerator ResetScene()
{
    yield return new WaitForSeconds(4f);
    _gameEngineScript.ResetScene();
}
```